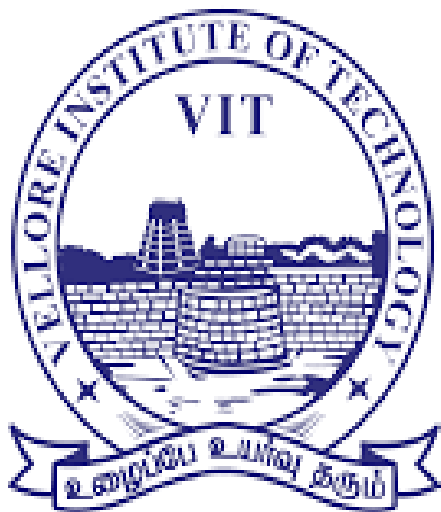




VIT[®]

BHOPAL

Student Study Planner

Soumya Tandon

25BSA10030

Project Report

1. Project Overview

A comprehensive Python-based Study Planner application that helps students organize their study schedule, track progress, and maintain motivation through gamification and analytics.

2. Introduction

□ Purpose:

The application serves as a digital companion for students who need better control over their study routines, offering tools to plan, monitor, and refine their academic activities.

□ Technology Stack:

The system uses Python in combination with visualization and data-handling modules like matplotlib, numpy, and built-in time-management and logging utilities.

□ Key Features:

Users can manage subjects, allocate study time, establish goals, record study sessions, evaluate progress, and stay motivated through a reward-based system.

□ Target Users:

Students who want consistent study habits and data-driven productivity tracking.

3. Problem Statement

Many students struggle to maintain steady study patterns and often lack a centralized method for organizing their workload. Without clear visual indicators of progress or systems that reward consistency, motivation tends to decline over time. Study logs are frequently fragmented or done manually, making them inefficient and inflexible. This project addresses these issues by providing a unified platform for scheduling, monitoring, and analyzing study efforts.

4. Functional Requirements

- - **Subject Management:** Allow users to add or remove subjects and specify time allocations.
- **Timetable Generation:** Produce daily or weekly study schedules based on user inputs.
- **Goal Setting:** Enable users to create and track academic goals for short-term and long-term durations.
- **Progress Logging:** Record study sessions with accurate timestamps for tracking patterns.
- **Motivation System:** Award points, maintain streaks, and unlock badges to encourage continued engagement.
- **Visual Analytics:** Create weekly study-time graphs using matplotlib for clear insights.
- **Performance Classification:** Examine study behavior and generate predictions of academic performance.
- **Data Handling:** Provide complete CRUD capabilities for study-related information.

- **Activity Logging:** Record all actions for transparency and review.

5. Non-functional Requirements

- **Performance:** The application must remain responsive even when handling large volumes of transactions.
- ☐ **Reliability:** The system should maintain high availability and recover smoothly from errors.
- ☐ **Security:** User inputs must undergo validation, and sensitive data must be managed securely.
- ☐ **Usability:** The console interface should be intuitive and easy for students to navigate.
- ☐ **Maintainability:** The codebase should be modular and well-documented to simplify future updates.
- ☐ **Scalability:** The system should accommodate significant growth in stored financial records.

6. System Architecture

...

The application is built on the MVC structure to maintain clear separation of concerns. A layered architecture divides the system into presentation, business logic, and data-access

layers. The repository pattern abstracts storage operations, making the system adaptable to more advanced databases if needed. Strategy pattern usage allows prediction algorithms to be swapped or extended with minimal code changes. The entire system is organized into modular components that can be easily tested and improved.

7. Design Diagrams

Use Case Diagram

'''

Actors: Student

Use Cases:

- Manage Subjects (Add/Delete)
- Allocate Study Time
- Set Academic Goals
- Log Study Sessions
- View Progress Analytics
- Earn Rewards/Badges
- Generate Visual Reports
- Predict Performance

'''

Workflow Diagram

...

Start → Main Menu → Function Selection → Data
Processing →

Result Display → Log Activity → Return to Menu → Exit

...

Sequence Diagram

...

User → System: Select Option

System → Function: Execute Command

Function → Data: Process Request

Data → Function: Return Result

Function → Visualization: Generate Graph (if needed)

System → User: Display Output

System → Logger: Record Activity

...

Class/Component Diagram

...

Main Components:

- Menu Controller (main())
- Subject Manager
- Goal Manager

- Session Logger
- Reward System
- Analytics Engine
- Visualization Generator
- Activity Logger

'''

ER Diagram (In-memory Storage)

'''

Subjects: [name]

TimeAllocation: {subject: hours}

Goals: {daily: [], weekly: []}

Sessions: [{subject, hours, date, time}]

UserStats: {points, streak, badges}

'''

8. Design Decisions & Rationale

A console-based interface was chosen to keep the system lightweight and compatible across platforms. JSON storage provides a simple yet effective means of saving data without requiring database installation. Implementing the strategy pattern enables flexible additions to the prediction engine, while the repository pattern prepares the system for scaling to advanced storage solutions. Custom exception handling ensures clearer feedback and smoother error resolution.

9. Implementation Details

The system is built using Python 3.8+ and follows an MVC architectural structure with layered separation. User data is stored in JSON files for persistence. Security practices include hashing credentials with SHA-256 and sanitizing all inputs. Prediction features incorporate techniques such as moving averages and linear regression. The solution also uses several key design patterns—Factory, Strategy, Repository, and MVC—to maintain structure and extensibility.

10. Testing Approach

-Testing includes unit tests for individual modules, integration tests to evaluate end-to-end flow, validation tests for input boundaries, and stress tests for handling large datasets. Error-handling tests ensure the system gracefully manages invalid operations or unexpected situations.

12. Challenges Faced

It was important to balance simplicity with providing substantial functionality. Generating accurate spending forecasts with limited data proved difficult. Ensuring reliable data persistence without relying on a full-scale database introduced design constraints. Creating informative, user-friendly error messages also required careful consideration. As transaction counts grow, maintaining speed and responsiveness became another area of focus.

13. Learnings & Key Takeaways

- Using design patterns enhanced clarity and maintainability of the code. Strong validation processes minimized runtime errors. Even in console applications, user experience plays a major role in overall effectiveness. Modular architecture allowed new features to be integrated quickly, and extensive logging greatly simplified debugging and long-term maintenance.

14. Future Enhancements

Possible upgrades include a web-based interface, mobile applications for easier access, and cloud support for multi-device syncing. Additional improvements could involve advanced analytics powered by machine learning, expanded export options, multicurrency support, API-based banking integration, and shared budgeting tools for families or groups.

15. References

- Python Official Documentation
- matplotlib User Guide
- Software Engineering Principles